

10월 9일 — RAG 실험의 날

대상: 팀원 4명(PM · A · B · C) · 함께 보는 문서: 「진행 현황과 구현 목표」(10월 8일 기준)

오늘의 목표

- 네 명 모두 RAG 한 단계씩 직접 실험한다. 폴더 담당(오너)은 지금처럼 두고, 실험은 각자 브랜치에서 한다.
- 같은 기준으로 비교해 가장 좋은 설정을 고른다. 한 번에 하나만 바꾸고, 결과는 실험 기록표 한 장에 모은다.
- 마지막에 held-out v2로 딱 한 번詹다. 이 숫자가 발표에 들어가는 최종 성적이다.

1 역할 — 누가 RAG의 어느 단계를 맡나

사람	폴더 (오너, 그대로)	오늘 맡는 단계	할 일	배우는 것
A 법률 데이터 ·QA	data/tagged/ data/eval/	정답지 · 검색 설정	① held-out v2 11건 작성(오전, PM 검토) ② 검색 전용 질의 20~30개 작성(정답 조문만 적음) ③ 오후 실험: top-k, 검색 필터	Hit@3가 무엇을 기준으로 채점되는지, top-k와 후보 수의 관계
B 백엔드·Rule Engine	app/engine/ app/main.py · schemas.py · tests/	쿼리 변환 · 특성 추출	① 오전: LLM 특성 추출로 측정만(코드 수정 없음) ② 오후 실험: 영역 쿼리 문구(H02 공통 문구 제거 등) ③ 실패 수정: 특성 H01·H03·H07·H08, 규칙 H06·H09	일상어를 법률 용어 쿼리로 바꾸면 검색이 왜 좋아지는지
C RAG·평가	app/rag/ scripts/	임베딩 · 청크 표현 · 실험 운영	① 09:30 전: 준비 작업(3장) ② 오전 RAG 실습 진행, 임베딩 실험 ③ 오후 실험: 청크 앞 부모 맥락 등 ④ 조합 비교, 스냅샷 갱신, 실험 기록표 관리	임베딩 방식이 검색 결과를 어떻게 바꾸는지
PM 기획·프론트	app/static/ README.md · 기획안	결과 해석 · 설명	① held-out v2 검토(2인 검토) ② C와 함께 임베딩 실험 1개 직접 실험 ③ 화면의 '찾은 경로'·기각 후보가 실험 결과와 맞는지 점검 ④ 발표의 "왜 RAG인가" 파트를 실험 수치로 작성	RAG가 찾은 것과 규칙이 기각한 것을 남에게 설명하는 법

실험 브랜치에서는 남의 폴더 파일도 고쳐도 됩니다(예: B가 `app/rag/retriever.py` 의 쿼리 문구를 수정). **main에 넣을 때만** 그 폴더 오너가 리뷰하고 머지합니다.

2 하루 일정

시간	하는 일	끝나는 조건
09:30-10:00	킵오프 · 전원 기준선 재현. 각자 <code>ingest</code> → <code>evaluate</code> 를 돌려 수치가 <code>docs/eval_snapshot.json</code> 과 같은지 확인. 다르면 환경 문제부터 해결.	4명 모두 같은 수치
10:00-10:40	RAG 한 바퀴 실습 (C 진행). <code>data/processed/chunks.json</code> 에서 청크 몇 개를 직접 열어 보기 → H02(발전소)를 설명 그대로 vs 특성 쿼리로 검색해 결과 비교 → Hit@3를 손으로 계산해 보기.	모두 자기 단계가 파이프라인의 어디인지 설명 가능
10:40-12:30	A held-out v2 작성, PM 검토 · B LLM 특성 추출 측정(측정만) · C 임베딩 실험(PM이 1개 함께)	held-out v2 11건 확정

12:30	관문 — held-out v2 커밋 → 지금 held-out(v1)을 개발용 세트(dev2)로 이동. 이 전에는 누구도 held-out 실패를 고치지 않는다.	
13:30-15:30	단계별 실험(각자 브랜치) — A 검색 설정 · B 쿼리 문구 + 실패 수정 · C 체크 표현 · PM 화면 점검·발표 초안. 실험마다 기록표에 한 줄씩.	1인당 실험 2개 이상 기록
15:30-16:15	조합 비교 (C). 단계별 1등을 합친 조합 2~3개를 개발용 세트로 비교 → 최종 후보 2개.	최종 후보 2개와 고른 이유
16:15-16:45	held-out v2 측정 (딱 한 번). 최종 후보 2개만. C가 스냅샷 갱신, 결과표를 PM에게 전달. 결과가 나빠도 그대로 기록한다.	pytest 통과 · 발표용 결과표
16:45-17:30	각자 3분 공유 — "내 단계에서 무엇을 바꿨고, 어떤 케이스가 왜 좋아지거나 나빠졌나" · 데모 리허설	4명 공유 완료

3 준비 작업 — 실험을 빨리 돌리기 위해

지금 코드로도 환경 변수만 바꿔서 할 수 있는 실험이 있습니다. 다만 아래 세 가지 때문에 네 명이 동시에 실험하면 느리고 결과를 비교하기 어렵습니다. 09:30 전에 준비합니다(담당 C).

지금 상태	왜 문제인가	준비할 것
ingest 가 매번 Docling PDF 변환부터 다시 함	임베딩만 바뀌도 6개 PDF를 처음부터 다시 변환해 시간이 오래 걸린다	ingest --reuse-chunks : 이미 만든 chunks.json 으로 임베딩만 다시 하기
evaluate 결과가 results.json 하나에 덮어써지고, 항상 held-out까지 같이 계산함	실험 결과가 사라진다. 실험할 때마다 시험지(held-out) 점수를 보게 되어 고르는 과정에 섞인다	evaluate --split dev,dev2 --tag 실험이름 : 고른 세트만 계산하고 결과를 data/eval/runs/ 에 따로 저장
검색 정답 조문이 dev 18개뿐	정답 1개 차이로 Hit@3가 0.06 출렁여서 실험 간 차이가 실력인지 우연인지 구별이 안 된다	검색 전용 질의 세트 data/eval/retrieval_cases.json (A 작성) + 이 세트는 ① 검색 지표만 계산

코드 수정 없이 바꿀 수 있는 설정 (환경 변수)

변수	값	무엇이 바뀌나	다시 적재?
EMBEDDING_PROVIDER	local · openai	임베딩 방식	필요
EMBEDDING_MODEL	text-embedding-3-small · -large	OpenAI 임베딩 모델	필요
COLLECTION_NAME	exp_이름_설정	Chroma 컬렉션 이름. 실험마다 따로 써서 기준선을 덮어쓰지 않기	—
RETRIEVAL_TOP_K	3 · 5 · 8 · 10	쿼리당 가져오는 후보 수 (Hit@3는 그대로, 후보 재현율과 ②이 바뀜)	불필요
FEATURE_EXTRACTOR	heuristic · llm	특성 추출기	불필요

합정 — API 키를 넣으면 두 가지가 동시에 바뀐다

.env 의 FEATURE_EXTRACTOR 기본값은 auto 라서, OpenAI 임베딩 실험을 하려고 OPENAI_API_KEY 를 넣으면 특성 추출도 자동으로 LLM으로 바뀝니다. 임베딩 실험에서는 반드시 FEATURE_EXTRACTOR=heuristic 으로 고정하세요. 그래야 임베딩 효과만 볼 수 있습니다.

4 실험 규칙 — 공정하게 비교하려면

1. 기준선부터. 같은 세트로 기준선(현재 설정)을 먼저 재고, 모든 실험은 기준선과 비교한다.

2. 한 번에 하나만 바꾼다. 두 개를 같이 바꾸면 무엇 때문에 좋아졌는지 알 수 없다.
3. 고르는 건 개발용 세트로만. dev 11 + dev2 11(웁긴 v1) + 검색 전용 질의. held-out v2는 16:15에 최종 후보 2개만 한 번 쟀다.
4. 후보 수도 같이 본다. top-k를 늘리면 재현율이 오르는 건 당연하다. 같은 후보 수에서 비교한다.
5. 정답 1~2개 차이는 같은 성능으로 본다. 그럴 땐 더 싸고 단순한 쪽(로컬 임베딩, 작은 top-k)을 고른다.
6. 숫자만 적지 않는다. 결과가 바뀐 케이스 하나를 `results.json` 에서 열어 왜 바뀌었는지 한 줄 적는다. 오늘의 진짜 목적은 학습이다.
7. 스냅샷(`--snapshot`)은 C만 갱신한다. 여러 명이 갱신하면 서로 충돌한다.

5 실험 메뉴

단계	담당	바꿔 볼 것	볼 지표
임베딩	C · PM	로컬 해싱(기준) → OpenAI <code>text-embedding-3-small</code> → <code>-large</code> . 환경 변수와 새 컬렉션으로 다시 적재.	Hit@3 설명 그대로 · Hit@3 특성 쿼리
검색 설정	A	top-k 3 / 5 / 8 / 10 · 검색 필터에 BKL 원문 청크 포함(<code>retriever.py</code> 의 <code>SEARCH_FILTER</code>)	후보 재현율 vs 평균 후보 수 · ㉠ 최종
쿼리 변환	B	영역 쿼리의 공통 문구("고영향 인공지능 영역") 빼기 · 쿼리 문구 바꾸기 (<code>PURPOSE_QUERY</code>) · LLM 특성 추출	Hit@3 특성 쿼리 · ㉠ 최종
청크 표현	C	청크 앞 부모 맥락(상위 조문 제목) 붙이기 / 빼기 · 조문 제목 포함 여부	Hit@3 두 가지 · 청크 수
조합	C	단계별 1등을 합친 조합 2~3개	㉠ 최종 · 케이스 완전 일치

청크 실험 주의

정답지는 청크 ID(`ARTICLE_2_4_GA` 같은 이름)로 채점합니다. 청크를 자르는 단위를 바꿔 ID가 달라지면 채점이 깨지므로, 오늘은 ID는 그대로 두고 청크에 들어가는 글만 바꿉니다.

6 명령어 (Windows PowerShell)

```
# 0) 기준선 - 전원, 09:30
python -m scripts.ingest
python -m scripts.evaluate

# 1) 임베딩 실험 - 새 터미널에서 (환경 변수는 그 터미널에만 적용)
$env:FEATURE_EXTRACTOR = "heuristic"          # 함정 방지: 특성 추출 고정
$env:EMBEDDING_PROVIDER = "openai"
$env:EMBEDDING_MODEL = "text-embedding-3-small"
$env:COLLECTION_NAME = "exp_emb_openai_small"
python -m scripts.ingest                      # 새 컬렉션에 적재 (기준선 law_articles는 그대로)
python -m scripts.evaluate

# 2) top-k 실험 - 다시 적재할 필요 없음
$env:RETRIEVAL_TOP_K = "8"
python -m scripts.evaluate

# 3) LLM 특성 추출 측정 - .env에 OPENAI_API_KEY 필요
$env:FEATURE_EXTRACTOR = "llm"
python -m scripts.evaluate

# 결과 보관 - evaluate는 매번 results.json을 덮어쓴다
New-Item -ItemType Directory -Force data\eval\runs | Out-Null
Copy-Item data\eval\results.json data\eval\runs\emb_openai_small.json

# 실험이 끝나면 터미널을 닫거나 환경 변수 지우기
Remove-Item Env:EMBEDDING_PROVIDER, Env:EMBEDDING_MODEL, Env:COLLECTION_NAME, Env:RETRIEVAL_TOP_K,
Env:FEATURE_EXTRACTOR -ErrorAction SilentlyContinue
```

준비 작업(3장)이 끝나면 결과 보관은 `python -m scripts.evaluate --split dev,dev2 --tag emb_openai_small` 한 줄로 바뀝니다. 같은 터미널에서 적재와 평가를 해야 임베딩 설정이 맞습니다.

검색 전용 질의 형식 (A 작성)

```
[
  { "id": "R01",
    "spec": "변전소 전력 수요를 예측해 공급량을 자동으로 조절하는 AI",
    "gold": { "retrieval": ["ARTICLE_2_4_GA"] } }
]
```

판정 정답(statuses) 없이 "이 문장이면 이 조문이 검색돼야 한다"만 적습니다. 고정형 영역(가~카목)마다 표현을 바꾼 문장 2~3개씩 쓰면 금방 20~30개가 됩니다.

7 실험 기록표 — 공유 시트 한 장 (C가 관리)

ID	담당	바꾼 것 (하나)	설정 · 브랜치	평균 후보	Hit@3 설명	Hit@3 특성	후보 재현율	◎ 최종	판단	바뀐 케이스와 이유
E00	C	기준선	local · top-k 5	26.8	0.28	0.83	0.89	1.00	기준	dev 11 기준 예시. dev2 이동 후 다시 잼
E01										
E02										

판단은 **채택** **보류** **기각** 중 하나. 기각된 실험도 지우지 않습니다 — 발표에서 "해 봤는데 안 됐다"도 근거가 됩니다.

8 협업 규칙

- **브랜치:** 실험은 `exp/이름-실험` (예: `exp/a-topk`). main에는 채택된 것만, 폴더 오ner가 리뷰 후 머지.
- **held-out v1 이동:** v1은 `cases.json`에 합치지 않고 `data/eval/dev2_cases.json`으로 따로 둔다. 그래야 기존 dev 11건의 "회귀 1.0 = 머지 기준"이 그대로 유지된다. dev2는 고치는 대상이라 머지 기준에서 뺀다.
- **정답이 의심되면** 고치지 말고 A에게. A가 근거를 정리하고 PM이 결정, 판정 결정 로그에 남긴다.
- **막히면 30분 안에 말하기.** 혼자 붙잡고 있는 게 가장 큰 일정 리스크다.

9 오늘 끝났다고 보는 기준

✓	항목	담당
☐	held-out v2 11건 커밋 (2인 검토)	A · PM
☐	검색 전용 질의 20개 이상	A
☐	실험 기록표 — 1인당 실험 2개 이상, 각 실험에 "바뀐 케이스와 이유"	전원
☐	최종 설정 결정과 고른 이유 (단계별 1등 + 조합 비교)	C
☐	held-out v2 1회 측정, 스냅샷 갱신, pytest 통과	C · B
☐	발표용 결과표 (기준선 → 최종) 와 "왜 RAG인가" 장면	PM
☐	각자 3분 공유	전원

근거: docs/roles.md(폴더 오ner·협업 규칙) · docs/evaluation.md(지표·실패 분석) · app/config.py(환경 변수) · scripts/ingest.py · scripts/evaluate.py.
3장의 준비 작업은 아직 코드에 없습니다.